

Republic of the Philippines

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

College of Computer and Information Sciences

Sta. Mesa, Manila

LAlgo

Language Design Specification

A Domain-Specific Language for Observable Execution,
Tracing, and Reasoning in Algorithms

Formal Syntax and Operational Lexicon

Submitted by: Group 4

Buenaventura, Harvey L.

De Mesa, Joshua C.

Dela Rosa, Mico Von Adriel A.

Magalona, Adriel M.

Morales, Timothy Gabriel

Nevado, Sean Andrew M.

Salvacion, Dean James G.

Submitted to

Prof. Ria A. Sagum

Course and Subject Code

Principles of Programming Languages (COSC 303)

Contents

1 Introduction **1**
- 1.1 Overview . . . 1
- 1.2 Inspiration . . . 1
- 1.3 Problem Addressed . . . 1
- 1.4 Objectives . . . 2

2 Lexical Elements **2**
- 2.1 Character Set . . . 2
- 2.2 Identifiers . . . 3
- 2.3 Finite Automata (Lexical Analyzer Execution Model) . . . 3
  - 2.3.1 Unified DFA with One Start State . . . 4
  - 2.3.2 DFA for Identifiers and Keywords . . . 5
  - 2.3.3 DFA for Numeric Literals (Integer and Decimal) . . . 5
  - 2.3.4 DFA for Relational and Assignment Operators . . . 6
  - 2.3.5 DFA for Comments and Division . . . 7
  - 2.3.6 DFA for String Literals . . . 7
  - 2.3.7 DFA for Character Literals . . . 8
  - 2.3.8 DFA for Whitespace (Blanks) . . . 8
  - 2.3.9 DFA for Delimiters and Brackets . . . 8
  - 2.3.10 Representative DFA for Keyword Recognition . . . 9
  - 2.3.11 Scope and Limitations . . . 9
  - 2.3.12 Token Structure and Error Handling . . . 10
  - 2.3.13 Implementation Input, Output, and Demonstration . . . 11
- 2.4 Operation Symbols . . . 11
  - 2.4.1 Arithmetic Operators . . . 11
  - 2.4.2 Boolean (Logical) Operators . . . 12
  - 2.4.3 Relational Comparison Operators . . . 12
- 2.5 Keywords and Reserved Words . . . 13
- 2.6 Noise Words . . . 13
- 2.7 Comments . . . 13
  - 2.7.1 Single-Line Comments . . . 14
  - 2.7.2 Multi-Line Comments . . . 14
- 2.8 Blanks (Whitespace Handling) . . . 14
- 2.9 Delimiters and Brackets . . . 14
- 2.10 Free-Field Format . . . 15
- 2.11 Expressions . . . 15
  - 2.11.1 Operator Precedence and Associativity . . . 16
- 2.12 Primitive Data Types . . . 16
  - 2.12.1 Integer . . . 16
  - 2.12.2 Decimal . . . 16
  - 2.12.3 Character . . . 16
  - 2.12.4 Boolean . . . 16
  - 2.12.5 Abstract Data Types (Optional / Supported) . . . 17
- 2.13 Statements . . . 17
  - 2.13.1 Declarative Statement (Optional) . . . 17
  - 2.13.2 Input Statement . . . 17
  - 2.13.3 Output Statement . . . 17
  - 2.13.4 Assignment Statement . . . 17
  - 2.13.5 Conditional Statement . . . 18
  - 2.13.6 Loop (Iterative) Statements . . . 18
  - 2.13.7 Future Reserved Extensions . . . 18

A Worked Binary-Search Token Stream **19**

B Complete Unified DFA Transition Specification **21**
- B.1 All Transitions from the Single Start State . . . 21
- B.2 All States, Accepting Actions, and Remaining Transitions . . . 22

1 Introduction

1.1 Overview

LAlgo is a proposed imperative domain-specific language (DSL) for expressing data structures and algorithms (DSA) in readable pseudocode. This proposal focuses on the lexical analyzer: the character alphabet, token classes, and one unified deterministic finite automaton used to recognize source text. A complete parser and runtime are future work.

The examples use familiar declarations, operators, indexing, and control-flow words to show the intended token vocabulary. They illustrate how an algorithm such as binary search could be written, but they do not claim that execution tracing or invariant checking is implemented by the present lexical proposal.

1.2 Inspiration

The primary inspiration for LAlgo emerges from the cognitive disconnect students encounter when transitioning between algorithmic pseudocode and practical code implementation. Textbooks routinely present algorithms alongside step-by-step state trace tables and formal loop assertions. However, translating these designs into standard programming environments forces students to manage language-specific boilerplate, unmonitored variable states, and implicit type conversions that mask algorithmic errors.

When a learner encounters an off-by-one error, the sequence of intermediate states can be difficult to reconstruct. LAlgo draws inspiration from educational pseudocode, Pascal-like syntax, and structured tracing tools. This paper establishes the lexical groundwork for that longer-term direction without specifying a tracing runtime.

1.3 Problem Addressed

Traditional programming language compilers and runtime engines are intentionally opaque: they consume source code, optimize away intermediate variables, and produce final output. Under this execution model:



This pipeline introduces several core educational problems:

- **Silent Intermediate States.** Students observe only terminal outputs, leaving internal invariant breakdowns, element swaps, and path traversals invisible.
- **Unverified Invariants.** Algorithmic correctness proofs rely on inductive loop invariants, yet standard languages provide no native, readable syntactic construct to express whether loop conditions remain valid across iterations.
- **Hidden State Changes.** Intermediate variable updates and branch decisions can be difficult to follow without manually adding diagnostic output.

The present proposal addresses the lexical foundation of this problem. Parsing, execution-state observation, and any later tracing or verification behavior require separate specifications.

1.4 Objectives

The primary goal of this iteration is to define an unambiguous token vocabulary and deterministic lexical recognition model for algorithmic source text. Specifically, the project aims to:

1. Define an unambiguous lexical structure and formal automata suited for algorithmic problem solving.
2. Recognize tokens used by illustrative declarations, I/O, assignments, conditional branches, and loops without defining their full parser or runtime behavior.
3. Recognize collection-type words (`ARRAY` and `STRING`) and the associated delimiters.
4. Reserve `ALGORITHM`, `RETURN`, `TRACE`, and `INVARIANT` for possible future syntactic extensions, without assigning them executable behavior in this iteration.

2 Lexical Elements

2.1 Character Set

LAlgo utilizes a standard ASCII-based character set denoted by the finite alphabet Σ. The alphabet is organized into lexical classes:

$$\Sigma = \Sigma_{\text{alpha}} \cup \Sigma_{\text{digits}} \cup \Sigma_{\text{arithmetic}} \cup \Sigma_{\text{relational}} \cup \Sigma_{\text{delimiters}} \cup \Sigma_{\text{underscore}} \cup \Sigma_{\text{quotes}} \cup \Sigma_{\text{whitespace}} \cup \Sigma_{\text{literal}} \cup \Sigma_{\text{comment}}.$$

The supported character categories are explicitly defined below.

Alphabetic Characters (Σ_{alpha}) `a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`

Numeric Digits (Σ_{digits}) `0 1 2 3 4 5 6 7 8 9`

Arithmetic Symbols ($\Sigma_{\text{arithmetic}}$) `+ - * / % ^`

Relational and Assignment Symbols ($\Sigma_{\text{relational}}$) `= ! < >`

Delimiters and Punctuation ($\Sigma_{\text{delimiters}}$) `( ) [ ] { } , ; : .`

Identifier Continuation Symbol ($\Sigma_{\text{underscore}}$) `_`

Quotation Symbols (Σ_{quotes}) `" '`

Whitespace Characters ($\Sigma_{\text{whitespace}}$) Space (ASCII 32), horizontal tab (ASCII 9), and line feed (ASCII 10).

Literal/Comment-Only Symbols (Σ_{literal}) Printable ASCII 33–126 not already in another class and other than backslash; these symbols are permitted only inside quoted literals and comments.

Comment-Only Symbol (Σ_{comment}) Backslash (ASCII 92), permitted in comments only; it does not introduce an escape.

Space and horizontal tab are also permitted inside string literals, character literals, and comments, even though they belong to $\Sigma_{\text{whitespace}}$.

The scanner combines the individual relational characters into `==`, `!=`, `<=`, and `>=` using lookahead. These are multi-character operators rather than additional members of the character alphabet; `!` alone is not a valid operator.

2.2 Identifiers

Identifiers are programmer-defined tokens used to name variables, constants, and algorithmic routines. An identifier must satisfy the following lexical constraints:

- It must begin with an alphabetic character (`a-z` or `A-Z`).
- It may subsequently contain alphabetic characters, numeric digits (`0-9`), or underscores (`_`).
- It must not match any reserved keyword or noise word.
- It is strictly case-sensitive: `counter`, `Counter`, and `COUNTER` are distinct tokens.

Formally, the lexical structure of an identifier is expressed by the regular expression:

```
[a-zA-Z][a-zA-Z0-9_]*
```

Valid identifiers

```
low
high
binarySearch
target_index
arraySize2
```

Invalid identifiers

```
2low // Lexical error: number followed by
    letter
target-index // Hyphen means subtraction
WHILE        // Reserved keyword
SET          // Reserved noise word
```

2.3 Finite Automata (Lexical Analyzer Execution Model)

LAlgo's implemented lexical scanner processes raw source code using one unified deterministic finite automaton (DFA), with one start state and one shared transition table. Formally, the automaton is defined as a five-tuple:

$$M = (Q, \Sigma, \delta, q_0, F),$$

where Q is the finite set of internal states, Σ is the input character alphabet, $\delta : Q \times \Sigma \to Q$ is the state transition function, $q_0 \in Q$ is the initial state, and $F \subseteq Q$ is the set of accepting states.

Diagrams show only defined transitions; every missing transition goes to an implicit non-accepting error state, so δ is total. State sets listed below omit this implicit state for readability. The scanner remembers the last accepting position and leaves lookahead for the next token. Entering the error state ends a candidate match; it reports an error only if no valid token can be emitted or an explicit lexical restriction is violated. Once a quote or comment opener is recognized, an unterminated construct is an error, not a fallback to a shorter token.

2.3.1 Unified DFA with One Start State

The implementation in `lalgo_unified.c` builds one table, `DELTA[state][class]`, with 254 explicit states and 83 input classes. Every candidate lexeme starts at the same q_0. Keywords, identifiers, numeric literals, operators, comments, quoted literals, delimiters, and whitespace are paths within this single state graph. There is no runtime dispatch to separate recognizers and no post-scan string comparison for keywords.

The figure below is a grouped overview of the unified graph, not a substitute for its complete transitions. Each box denotes a state family; keyword letters and individual punctuation have distinct concrete states. Appendix B gives every state, accepting action, and defined transition directly from the C implementation. The category diagrams that follow use local state labels as explanatory projections of these paths; they are not independently executed DFAs and their local q_0 denotes the same global start.

After each transition the scanner remembers the last accepting state and source position. It emits the longest accepted prefix, retains unused lookahead, then restarts at q_0 for the next lexeme. Whitespace accepts with a skip action. A malformed-number state accepts an INVALID candidate for diagnostic reporting. Unterminated quoted literals and block comments use committed-state metadata to prevent fallback to a shorter token. These scanner actions are distinct from DFA transitions.

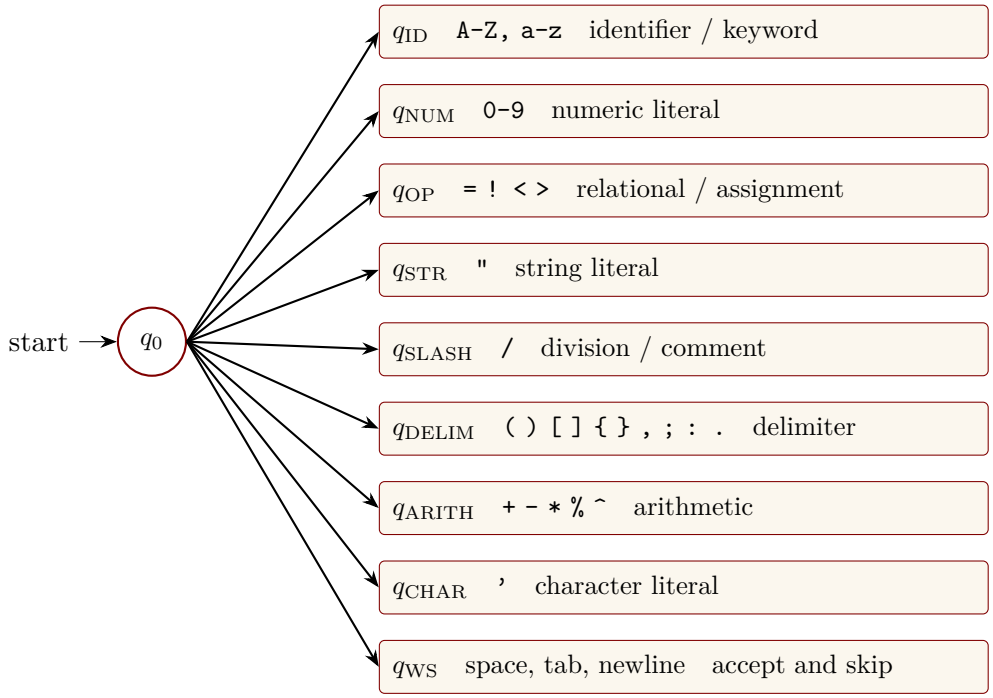


Figure 1: Grouped overview of one unified DFA: all token families branch from the single start state q_0.

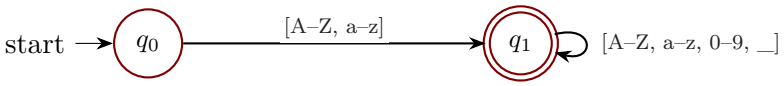
Step-by-step lexical scan trace. Consider the statement `count = 10`.

Step	Current state	Character(s)	Automaton action	Token emitted
1	q_0	`'c'`	Transition to q_{ID}	None
2	q_{ID}	`'o', 'u', 'n', 't'`	Self-loop on q_{ID}	None
3	q_{ID}	space	No accepting extension; emit and reset	`TOKEN(IDENT, "count")`
4	q_0	space	Consume whitespace recognizer	None
5	q_0	`'='`	Transition to q_{OP}	None
6	q_{OP}	space	Lookahead is not `'='`; emit and reset	`TOKEN(ASSIGN, "=")`
7	q_0	space	Consume whitespace recognizer	None
8	q_0	`'1'`	Transition to q_{NUM}	None
9	q_{NUM}	`'0'`	Self-loop on q_{NUM}	None
10	q_{NUM}	EOF	Buffer end; emit and reset	`TOKEN(INTEGER, 10)`

2.3.2 DFA for Identifiers and Keywords

Identifiers begin with an alphabetic character and may continue with letters, digits, or underscores. This diagram projects the general identifier shape. In the full unified DFA, uppercase keyword-prefix states accept their associated token at complete spellings; a non-keyword continuation falls into the general identifier state. Lowercase or mixed-case prefix states accept IDENT and flag a complete keyword spelling as suspected. Keyword recognition is part of the same scan, not a later lookup.

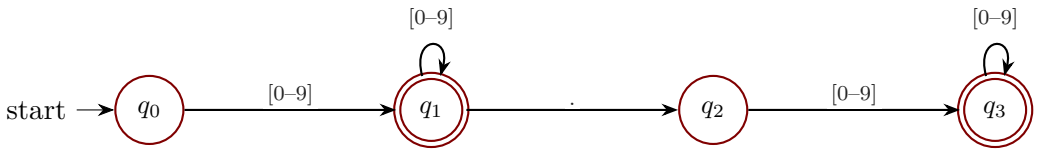
- $Q = \{q_0, q_1\}$
- q_0: initial state
- $F = \{q_1\}$ (accepting state)



2.3.3 DFA for Numeric Literals (Integer and Decimal)

This automaton distinguishes whole-number values (**INTEGER**) from fractional real numbers (**DECIMAL**) through decimal-point transitions.

- $Q = \{q_0, q_1, q_2, q_3\}$
- $F = \{q_1, q_3\}$, where q_1 accepts **INTEGER** and q_3 accepts **DECIMAL**



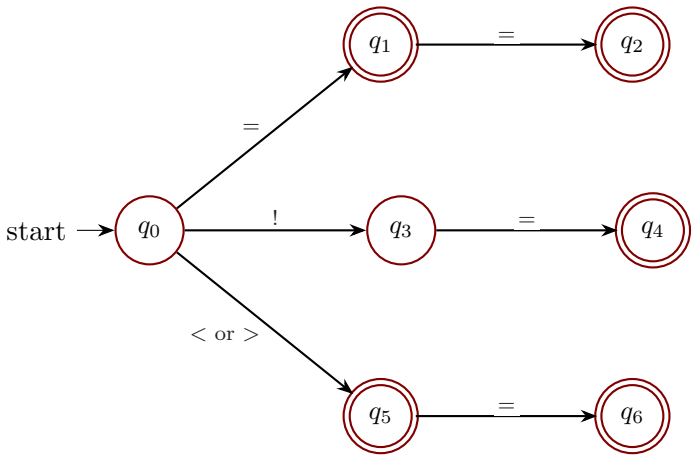
- **Trailing dot.** `3.` is not a `DECIMAL`. From non-accepting q_2, the scanner backtracks to the last accepting position and emits `INTEGER 3`, followed by `DOT`.

- **Leading dot.** `.5` is `DOT` followed by `INTEGER 5`. A decimal requires at least one digit on each side of the point.
- **Leading zeros.** Allowed and ignored in value: `007` = 7 and `00.5` = 0.5. The original spelling remains in the token's lexeme.
- **Range.** Integer literals must fit the signed 32-bit range; their maximum unsigned magnitude is 2147483647. Larger literals are lexical errors. Since signs are separate operators, even the magnitude 2147483648 in `-2147483648` is rejected. Double precision is the proposed runtime representation for decimals; this lexer recognizes decimal spelling without converting values or checking floating-point range.

2.3.4 DFA for Relational and Assignment Operators

This DFA disambiguates single-character assignment and comparison operators (`=`, `<`, `>`) from compound relational operators (`==`, `!=`, `<=`, `>=`).

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$
- $F = \{q_1, q_2, q_4, q_5, q_6\}$

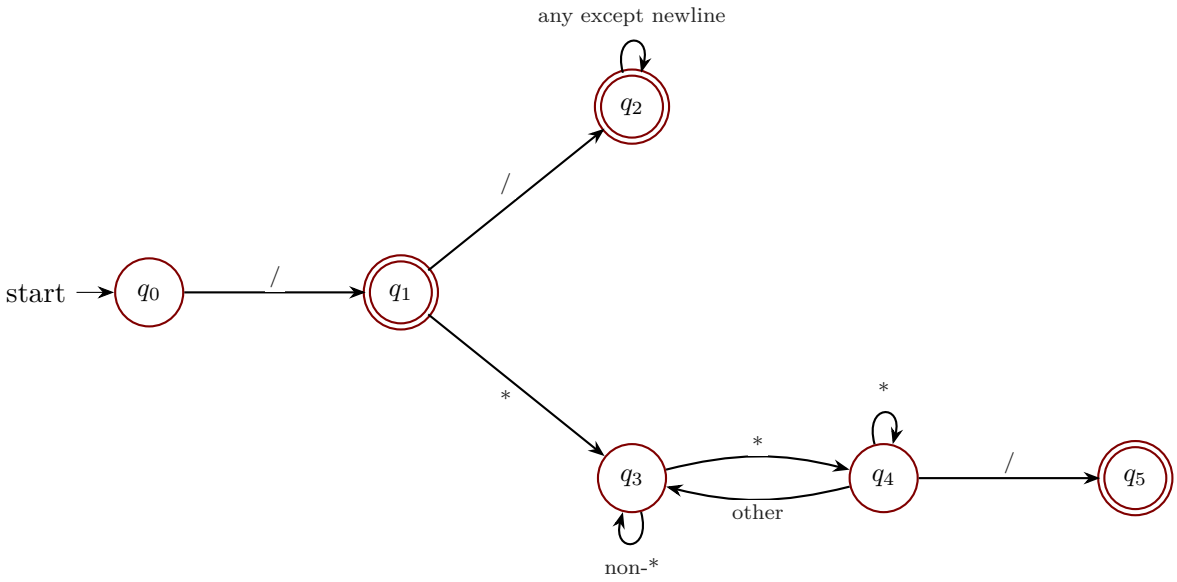


2.3.5 DFA for Comments and Division

Transition labels such as “other” and “non-*” range only over the permitted comment-body characters in Section 2.7; unsupported characters are lexical errors.

This DFA distinguishes division (`/`) from single-line comments (`//`) and block comments (`/* ... */`) using lookahead scanning.

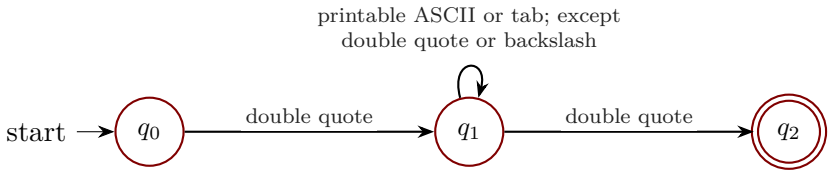
- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$
- $F = \{q_1, q_2, q_5\}$, where q_1 accepts division, q_2 accepts a single-line comment, and q_5 accepts a multi-line comment



2.3.6 DFA for String Literals

This DFA recognizes character-string literals enclosed by double quotation marks (`"..."`). Their contents consist of printable ASCII characters other than a double quote or backslash, plus horizontal tab; escape sequences and embedded newlines are not defined.

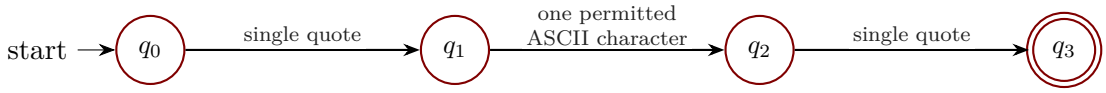
- $Q = \{q_0, q_1, q_2\}$
- $F = \{q_2\}$



2.3.7 DFA for Character Literals

A character literal consists of one printable ASCII character or horizontal tab enclosed in single quotation marks, such as `'A'` or `','`. The character between the quotes may be ASCII 32–126 except a single quote or backslash, or a horizontal tab; escape sequences are not defined in this specification. An empty, multi-character, or unterminated literal is a lexical error.

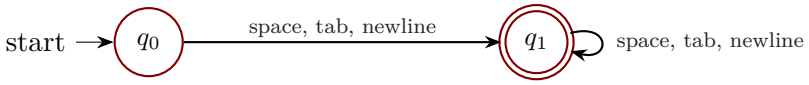
- $Q = \{q_0, q_1, q_2, q_3\}$
- $F = \{q_3\}$



2.3.8 DFA for Whitespace (Blanks)

Whitespace characters—spaces, horizontal tabs, and line feeds—are recognized by a trivial single-loop automaton. Tokens matched by this automaton are consumed by the lexer but are not emitted to the parser; they separate adjacent lexemes and update source positions and newline metadata.

- $Q = \{q_0, q_1\}$
- $F = \{q_1\}$ (accepting, but discarded)

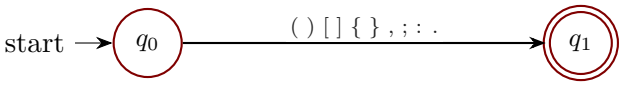


Because whitespace carries no semantic weight (Section 2.8), the scanner discards any accepted whitespace lexeme instead of forwarding it to the parser, preserves any line-feed information in token metadata, then resumes scanning from q_0.

2.3.9 DFA for Delimiters and Brackets

Delimiters and brackets are single-character tokens: `( ) [ ] { } , ; : .` . Because no lookahead is required, each is accepted after exactly one transition from q_0.

- $Q = \{q_0, q_1\}$
- $F = \{q_1\}$, where the specific delimiter determines the emitted token type (for example, `LPAREN`, `RBRACE`, or `COMMA`)



Unlike the multi-character automata above, no maximal munch is needed: the lexer emits the corresponding token immediately after the single accepted transition and resets to q_0.

2.3.10 Representative DFA for Keyword Recognition

The identifier DFA defines the shape shared by identifiers and reserved words: a letter followed by zero or more letters, digits, or underscores. The automaton below illustrates how that general shape narrows into the exact spelling of `IF`, `INPUT`, `INTEGER`, and `INVARIANT`. The first three are current language keywords; `INVARIANT` is reserved for a future extension. They share the prefix `I` and diverge as their spellings differ.

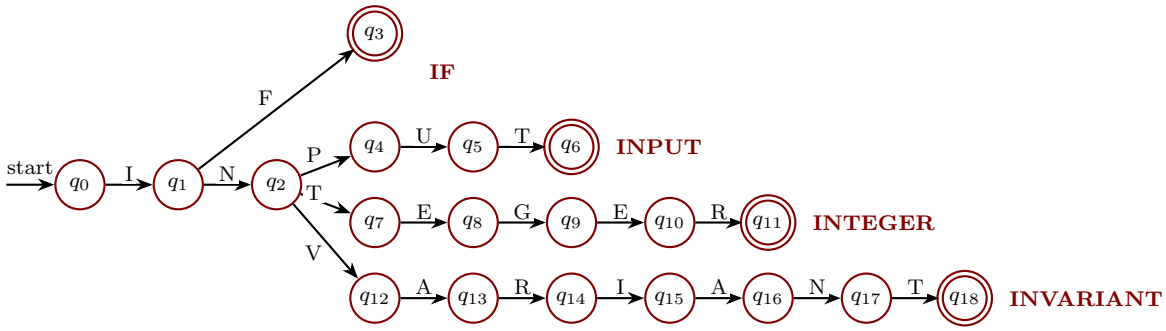


Figure 2: Shared-prefix keyword recognition with non-overlapping branches and explicit accepting states.

At each accepting state (q_3, q_6, q_{11}, and q_{18}), if the following character continues the identifier pattern `[a-zA-Z0-9_]` instead of a delimiter or whitespace, the scanner does not emit a keyword token. It falls back into the general identifier state q_{ID}. For example, `IFrame` does not match `IF` because `r` extends the identifier. Any unshown letter at a branch likewise routes to q_{ID}.

> **Note.** The same shared-prefix principle extends to other current keywords, future-reserved words, and noise words. For example, `ALGORITHM`, `ARRAY`, and `AND` share an A prefix, while `TRACE`, `TRUE`, and `THE` share a T prefix. The keyword trie is used only when constructing the unified table. During scanning, keyword letters are transitions in that same table. Exact-case states and mixed-case states are listed exhaustively in Appendix B; this diagram shows a representative uppercase projection.

2.3.11 Scope and Limitations

This finite-automata model operates strictly at the lexical level, where tokens are recognized and classified. It does not check type rules, declaration-before-use, statement structure, or grammar-level correctness. Those checks belong to parsing and semantic analysis.

Several simplifications keep the model clear and within the scope of lexical analysis:

- **Error handling.** The error state is implicit. An unterminated string literal, character literal, or block comment ends in a non-accepting state and is reported as a lexical error.
- **Keyword resolution.** The keyword diagram is a projection of the unified table. Exact uppercase prefixes and prefixes containing lowercase letters have separate states. Digits, underscores, and non-matching letters lead to the general identifier state; no separate runtime keyword lookup is performed.
- **Future-reserved words.** `ALGORITHM`, `RETURN`, `TRACE`, and `INVARIANT` are recognized as reserved lexemes but have no grammar or runtime semantics in this iteration.
- **Nested brackets.** At the lexical level, `{`, `}`, `(`, `)`, `[`, and `]` are individual delimiter tokens. Nesting and matching are parser responsibilities.
- **Newlines.** CRLF line endings are normalized to a single line feed before lexical scanning; an isolated carriage return is not accepted.
- **Block comments.** Block comments do not nest; the first `*/` closes the comment.

- **Quotes inside literals.** Because escape sequences are not defined, a string cannot contain a double quote and a character literal cannot contain a single quote. Backslash is not permitted in either literal; escapes are future work.
- **Signed numbers.** Negative numbers are formed by a subtraction operator followed by a literal, so the numeric literal itself contains no sign.
- **Number-identifier boundaries.** A numeric literal immediately followed by an alphabetic character or underscore (for example, `5thSample`, `2low`, or `2_low`) is a lexical error: `Malformed number '2low'`. The scanner consumes the numeric candidate and the entire following run of `[A-Za-z0-9_]` as one bad lexeme and resumes after it. For example, `1.5abc` is one malformed number.

2.3.12 Token Structure and Error Handling

Every token is a record `(type, lexeme, line, column, precededByNewline)` with an additional Boolean `suspectedKeyword` flag (default `FALSE`). The lexeme preserves the exact source spelling, including quotation marks for quoted literals. Lines and columns start at 1 and identify the beginning of the token; a tab counts as one column. Positions refer to the source after CRLF normalization. EOF has an empty lexeme and the position immediately after the final source character.

Class	Token types
Identifier	`IDENT`
Literals	`INTEGER, DECIMAL, STRING, CHAR, TRUE, FALSE`
Type keywords	`KW_INTEGER, KW_DECIMAL, KW_CHARACTER, KW_BOOLEAN, KW_ARRAY, KW_STRING`
Control/I/O keywords	`KW_IF, KW_ELSE, KW_WHILE, KW_FOR, KW_INPUT, KW_OUTPUT`
Operator keywords	`DIV, AND, OR, NOT`
Arithmetic	`PLUS, MINUS, STAR, SLASH, PERCENT, CARET`
Relational / assignment	`ASSIGN, EQ, NEQ, LT, GT, LE, GE`
Delimiters	`LPAREN, RPAREN, LBRACKET, RBRACKET, LBRACE, RBRACE, COMMA, SEMICOLON, COLON, DOT`
Noise	`NOISE_SET, NOISE_THE`
Future-reserved	`RES_ALGORITHM, RES_RETURN, RES_TRACE, RES_INVARIANT`
Comments	`LINE_COMMENT, BLOCK_COMMENT`
Invalid	`INVALID`
Special	`EOF`

Whitespace produces no token. Comments receive their own token entries (LINE_COMMENT and BLOCK_COMMENT). Invalid quoted literals, unterminated block comments, malformed numbers, out-of-range integers, illegal standalone characters, and lone `!` produce INVALID entries with line-and-column diagnostics. A terminated comment containing unsupported bytes retains its comment classification and receives diagnostics at those byte positions. Scanning continues after recovery; EOF is the last token for a source that passes input validation. Recovery uses these boundaries:

- Malformed numbers consume the whole bad numeric run described in Section 2.3.11; out-of-range integers consume their entire digit sequence.
- Invalid quoted literals consume through the matching closing quote, or stop before a line feed or at EOF if no closing quote occurs first. The next line remains available for scanning.

- An invalid comment consumes through its normal terminator (line feed/EOF for a line comment, first `*/` for a block comment). An unterminated block comment consumes to EOF. Unsupported characters inside comments are reported at their own positions.
- An illegal standalone character or lone `!` consumes one character. Skipped text still updates positions and newline metadata.

The earlier scan trace abbreviates token records. For `count = 10` on one line with no trailing newline, the complete records are:

```
(IDENT,   "count", 1,  1, FALSE)
(ASSIGN,  "=",     1,  7, FALSE)
(INTEGER, "10",    1,  9, FALSE)
(EOF,     "",      1, 11, FALSE)
```

Here and in the worked example, `suspectedKeyword` is **`FALSE`** unless explicitly stated.

2.3.13 Implementation Input, Output, and Demonstration

The executable accepts a source filename ending in `.lal` (case-insensitive extension); other extensions are rejected before scanning. It also rejects files containing NUL bytes. This is an input check, not a guarantee that renamed binary data is valid source: unsupported bytes are still subject to lexical diagnostics. Keywords remain case-sensitive regardless of the extension's case.

```
gcc -std=c99 -Wall -Wextra -o lalgo lalgo_unified.c
./lalgo sample.lal
./lalgo sample.lal t
```

The optional `t` enables a character-by-character unified DFA trace; normal operation prints tokens and errors without tracing. Console columns are line:column, token type, lexeme, newline metadata, and suspected-keyword metadata. The scanner writes `Symbol Table.txt` in the current working directory, replacing the previous output. Each non-EOF occurrence receives an entry containing its number, lexeme, token type, classification, and source position. Comments and INVALID entries are included. This required output is a lexical occurrence listing, not a declaration table with variable types, scopes, addresses, or runtime values. Comment and invalid displays escape line feeds and tabs and truncate long text; internal lexemes retain the source spelling.

Exit status 0 means no lexical errors, 1 means lexical errors were recorded, and 2 indicates usage or file input/output failure. Lexical success does not establish grammatical validity or execute the program.

2.4 Operation Symbols

LAlgo defines seven arithmetic operators, three core logical operators, and six relational comparison operators. The descriptions below indicate intended meanings; they do not constitute a runtime implementation.

2.4.1 Arithmetic Operators

LAlgo provides seven foundational arithmetic operations. `^` denotes exponentiation, following mathematical and textbook pseudocode notation; unlike C/Java, it is not bitwise XOR:

Operation	Operator	Associativity	Semantic description
Addition	`+`	Left	Computes the sum of operands.
Subtraction	`-`	Left	Computes the difference of operands.
Multiplication	`*`	Left	Computes the product of operands.
Division	`/`	Left	Computes a real quotient (`DECIMAL`).
Modulo	`%`	Left	Remainder after integer division.
Integer division	`DIV`	Left	Returns the truncated integer quotient.
Exponentiation	`^`	Right	Raises the base to a power.

Distinction between / and DIV:

```
INTEGER a = 7
INTEGER b = 2
OUTPUT a / b   // Yields Decimal: 3.5
OUTPUT a DIV b // Yields Integer: 3
OUTPUT a % b   // Yields Integer remainder: 1
```

Exponentiation example:

```
OUTPUT 2 ^ 3 // Yields 8
```

2.4.2 Boolean (Logical) Operators

Boolean expressions use three logical keywords. This lexical specification identifies their tokens; evaluation strategy, including any short-circuit behavior, is outside its scope.

Operation	Operator	Precedence	Semantic description
Logical NOT	`NOT`	Highest among logical operators	Unary negation; inverts the truth value.
Logical AND	`AND`	Medium	Conjunction; `TRUE` only if both operands are `TRUE`.
Logical OR	`OR`	Lowest	Disjunction; `TRUE` if at least one operand is `TRUE`.

2.4.3 Relational Comparison Operators

Comparison operators evaluate the quantitative relationship between expressions and yield a strictly typed `BOOLEAN` value (`TRUE` or `FALSE`).

Comparison	Symbol	Example
Equal to	`==`	`mid == target`
Not equal to	`!=`	`count != 0`
Less than	`<`	`low < high`
Greater than	`>`	`score > 75`
Less than or equal to	`<=`	`i <= length`
Greater than or equal to	`>=`	`val >= 0`

2.5 Keywords and Reserved Words

Reserved words are recognized by the lexer and cannot be used as identifiers. The current language keywords and the words held for future extensions are distinguished below. All use uppercase canonical spelling. Keywords are recognized only in their uppercase spelling. An identifier whose lowercase form equals the lowercase form of a reserved word (for example, `while` or `If`) is emitted as `IDENT` with `suspectedKeyword` set to `TRUE`, so later phases can report "Did you mean WHILE?". Such an identifier is not a lexical error and is not reserved. The same diagnostic rule applies to future-reserved and noise words; identifiers such as `input`, `string`, and `array` remain legal.

Category	Reserved words
Primitive types	`INTEGER`, `DECIMAL`, `CHARACTER`, `BOOLEAN`
Collection types	`ARRAY`, `STRING`
Control flow	`IF`, `ELSE`, `WHILE`, `FOR`
I/O statements	`INPUT`, `OUTPUT`
Boolean literals	`TRUE`, `FALSE`
Arithmetic operator	`DIV`
Logical operators	`AND`, `OR`, `NOT`
Future extensions (reserved only)	`ALGORITHM`, `RETURN`, `TRACE`, `INVARIANT`

The table contains 18 current language keywords and four future-reserved words. The latter are recognized as distinct tokens but are not valid executable constructs in this proposal. The separate noise words `SET` and `THE` are also reserved.

2.6 Noise Words

Noise words are optional lexical tokens designed to mirror natural-language pseudocode. They enhance readability for beginner programmers but carry no operational weight. A future parser may permit them in the illustrative positions below and omit them from its Abstract Syntax Tree (AST); this lexer recognizes them without enforcing statement positions. They remain reserved to prevent identifier ambiguity.

- `SET`: optional prefix before an assignment.

```
SET index = 0 // Semantically identical to: index = 0
```

- `THE`: optional readability marker in output routines.

```
OUTPUT THE result // Semantically identical to: OUTPUT result
```

2.7 Comments

Comments allow programmers to document algorithmic logic, state specifications, and preconditions. The implemented scanner recognizes comments and includes them in its console and Symbol Table output. A future parser may filter these comment entries. Comment bodies may contain any printable ASCII character (32–126), including backslash, and horizontal tab. Block comments may also contain line feeds. Any other character inside a comment is a lexical error. The regular expressions below describe comment boundaries subject to these character restrictions.

2.7.1 Single-Line Comments

Single-line comments begin with two forward slashes (`//`) and continue until the next newline.

Regular expression:

```
//[^\n]*
```

Example:

```
// Initialize the lower search boundary
INTEGER low = 0
```

2.7.2 Multi-Line Comments

Multi-line comments are delimited by `/*` and `*/`, allowing block documentation across multiple lines.

Regular expression:

```
/\*[\s\S]*?\*/
```

Example:

```
/*
Binary Search Implementation
Precondition: Input array must be sorted in ascending order.
Returns: Index of target element, or -1 if not found.
*/
```

2.8 Blanks (Whitespace Handling)

Whitespace includes spaces (ASCII 32), horizontal tabs (ASCII 9), and line feeds (ASCII 10). CRLF line endings are normalized to a single line feed before lexical scanning; an isolated carriage return is not accepted. LAlgo treats whitespace strictly as a non-semantic token separator:

- Whitespace demarcates adjacent keywords, identifiers, and literals.
- Consecutive whitespace characters collapse into one lexical boundary.
- Indentation carries no structural or scoping significance, preventing syntax errors caused by mixed tabs and spaces.

These lines are lexically equivalent:

```
INTEGER x = 100;
```

```
INTEGER       x    =    100;
```

2.9 Delimiters and Brackets

Delimiters define statement boundaries, parameter passing, collection bounds, and hierarchical scoping blocks.

- **Parentheses `( )`:** enclose mathematical expressions, conditional guards, and call-like constructs (future).

- **Square brackets `[ ]`:** delimit array literals and specify indexed element access.
- **Curly braces `{ }`:** delimit compound statement blocks, loops, and conditional branches; grouping and call-like constructs are future work.
- **Comma `,`:** separates arguments, parameters, and array elements.
- **Semicolon `;`:** is recognized as a separate token. Rules for using it as a statement separator belong to the future parser specification.

2.10 Free-Field Format

LAlgo uses a free-field format. Program source code is not bound to fixed columns, physical margins, or indentation rules.

- **Block markers.** Opening and closing braces are separate tokens in the examples; their nesting and scope effects belong to a future parser specification.
- **Statement formatting.** Outside comments and quoted literals, spaces, tabs, and line feeds separate tokens without defining statement boundaries. Whether tokens form one or multiple statements is a parser-level question.

Line feeds are consumed and never emitted as separate tokens. For ordinary tokens and EOF, `precededByNewline` records whether a line feed occurred since the preceding ordinary token (or the start of the source), including within comments and recovered invalid text. Comment and INVALID entries are auxiliary: their own flag records a line feed since the preceding visible entry, but emitting them does not clear the pending flag for the next ordinary token. Thus the next ordinary token retains its newline information even when a comment is listed between statements. The future parser may use this metadata; it does not itself prescribe a statement-ending rule.

Illustrative free-field source:

```
INTEGER low = 0; INTEGER high = 10;
IF (low < high) { OUTPUT low; }
```

2.11 Expressions

The examples below show token sequences for intended arithmetic, relational, and logical expressions. Their complete grammar, typing, and evaluation rules are outside this lexical proposal.

1. **Arithmetic expressions** evaluate to numeric types (`INTEGER` or `DECIMAL`).

   ```
   (low + high) DIV 2
   ```

2. **Relational expressions** compare values to produce a truth condition.

   ```
   numbers[mid] > target
   ```

3. **Logical expressions** combine Boolean expressions using propositional connectives.

   ```
   (low <= high) AND (found == FALSE)
   ```

2.11.1 Operator Precedence and Associativity

Rank	Operator	Description	Associativity
1	`( ) [ ]`	Grouping and array indexing	Left-to-right
2	`^`	Exponentiation	Right-to-left
3	`-` (unary)	Sign inversion	Right-to-left
4	`* / %`, `DIV`	Multiplicative operations	Left-to-right
5	`+ -`	Additive operations	Left-to-right
6	`< <= > >=`	Relational inequalities	Left-to-right
7	`== !=`	Relational equalities	Left-to-right
8	`NOT`	Logical negation	Right-to-left
9	`AND`	Logical conjunction	Left-to-right
10	`OR`	Logical disjunction	Left-to-right

The table lists intended precedence from highest to lowest. In particular, `NOT found == TRUE` groups as `NOT (found == TRUE)`. These are requirements for the future parser, not operations performed by the lexer.

2.12 Primitive Data Types

LAlgo supports four fundamental primitive types and two built-in abstract data types suited for algorithmic collections.

2.12.1 Integer

Represents whole numbers: positive, negative, or zero.

```
INTEGER count = 0
INTEGER step = -5
```

2.12.2 Decimal

Represents numbers containing fractional values.

```
DECIMAL ratio = 0.618
DECIMAL total = -12.50
```

2.12.3 Character

Represents one printable ASCII character or horizontal tab enclosed in single quotation marks, following the character-literal DFA in Section 2.3.7.

```
CHARACTER grade = 'A'
CHARACTER delimiter = ','
```

2.12.4 Boolean

Represents binary truth values defined by the literals `TRUE` and `FALSE`.

```
BOOLEAN isFound = FALSE
BOOLEAN hasTerminated = TRUE
```

2.12.5 Abstract Data Types (Optional / Supported)

To support data-structure modeling, LAlgo provides two native collection types:

- **String.** An immutable ordered sequence of characters enclosed in double quotation marks.

```
STRING algorithmName = "Binary Search"
```

- **Array.** A contiguous, zero-indexed collection of homogeneous elements.

```
ARRAY numbers = [2, 4, 6, 8, 10]
INTEGER firstElement = numbers[0]
```

2.13 Statements

The following snippets illustrate token sequences for proposed declarations and procedural statements. They do not define a complete statement grammar or runtime semantics. The four future-reserved words are not used as executable statements here.

2.13.1 Declarative Statement (Optional)

Declarative statements use a type keyword followed by an identifier, optionally with an initializing expression. Default values for uninitialized variables are not specified by this lexical proposal.

```
INTEGER minIndex
DECIMAL threshold = 0.05
```

2.13.2 Input Statement

The `INPUT` statement is followed by a target identifier. Input conversion and runtime behavior are outside the scope of this lexical proposal.

```
INTEGER target
INPUT target
```

2.13.3 Output Statement

The illustrated `OUTPUT` form is followed by an expression. Output formatting and runtime delivery are not defined by the lexer.

```
OUTPUT "Search completed at index:"
OUTPUT targetIndex
```

2.13.4 Assignment Statement

The assignment symbol `=` is recognized between a target identifier or indexed value and an expression. Assignment evaluation is outside the lexical model.

```
low = mid + 1
numbers[0] = 99
```

2.13.5 Conditional Statement

Conditional statements provide two-way and multi-way branch selection through `IF`, optional `ELSE IF`, and `ELSE` blocks.

```
IF (numbers[mid] == target) {
    OUTPUT "Target Found"
    found = TRUE
}
ELSE IF (numbers[mid] < target) {
    low = mid + 1
}
ELSE {
    high = mid - 1
}
```

2.13.6 Loop (Iterative) Statements

The examples use `WHILE` and `FOR` tokens to illustrate indeterminate and determinate iteration forms.

WHILE loop (illustrative binary search). The `WHILE` form below shows a boundary update in both comparison directions. The `INVARIANT` token is reserved for future work and is intentionally absent from this example.

```
ARRAY numbers = [2, 4, 6, 8, 10]
INTEGER target = 8
INTEGER low = 0
INTEGER high = 4
INTEGER mid = 0
BOOLEAN found = FALSE
WHILE (low <= high AND found == FALSE) {
    mid = low + (high - low) DIV 2
    // Update search boundaries according to the comparison
    IF (numbers[mid] == target) {
        found = TRUE
        OUTPUT mid
    }
    ELSE IF (numbers[mid] < target) {
        low = mid + 1
    }
    ELSE {
        high = mid - 1
    }
}
```

FOR loop. The `FOR` construct packages counter initialization, loop-condition checking, and step updates into a single header.

```
FOR (INTEGER i = 0; i < 10; i = i + 1) {
    OUTPUT numbers[i]
}
```

2.13.7 Future Reserved Extensions

`ALGORITHM`, `RETURN`, `TRACE`, and `INVARIANT` are reserved lexemes only. This proposal does not define their parser productions, value rules, tracing schedule, invariant-failure behavior, or other runtime effects. They are intentionally omitted from the illustrative snippets above.

A Worked Binary-Search Token Stream

This is the complete lexical output for the 20-line `WHILE` binary-search example in Section 2.13.6, counting `ARRAY numbers` as line 1. Each indentation level uses four spaces. The closing brace on line 20 is followed by one line feed; EOF is therefore at line 21, column 1. The comment on line 9 is included as a LINE_COMMENT entry; its displayed text is shortened here to match the console-width limit. Positions, source lexemes, and newline flags are retained; every `suspectedKeyword` flag is `FALSE`.

The table is a worked lexical reference, not evidence of parser or runtime execution. Read rows from top to bottom. The last column is `precededByNewline`; an empty lexeme belongs only to EOF.

Line:column	Type	Lexeme	Newline before
1:1	KW_ARRAY	ARRAY	FALSE
1:7	IDENT	numbers	FALSE
1:15	ASSIGN	=	FALSE
1:17	LBRACKET	[	FALSE
1:18	INTEGER	2	FALSE
1:19	COMMA	,	FALSE
1:21	INTEGER	4	FALSE
1:22	COMMA	,	FALSE
1:24	INTEGER	6	FALSE
1:25	COMMA	,	FALSE
1:27	INTEGER	8	FALSE
1:28	COMMA	,	FALSE
1:30	INTEGER	10	FALSE
1:32	RBRACKET	]	FALSE
2:1	KW_INTEGER	INTEGER	TRUE
2:9	IDENT	target	FALSE
2:16	ASSIGN	=	FALSE
2:18	INTEGER	8	FALSE
3:1	KW_INTEGER	INTEGER	TRUE
3:9	IDENT	low	FALSE
3:13	ASSIGN	=	FALSE
3:15	INTEGER	0	FALSE
4:1	KW_INTEGER	INTEGER	TRUE
4:9	IDENT	high	FALSE
4:14	ASSIGN	=	FALSE
4:16	INTEGER	4	FALSE
5:1	KW_INTEGER	INTEGER	TRUE
5:9	IDENT	mid	FALSE
5:13	ASSIGN	=	FALSE
5:15	INTEGER	0	FALSE
6:1	KW_BOOLEAN	BOOLEAN	TRUE
6:9	IDENT	found	FALSE
6:15	ASSIGN	=	FALSE
6:17	FALSE	FALSE	FALSE
7:1	KW_WHILE	WHILE	TRUE
7:7	LPAREN	(	FALSE
7:8	IDENT	low	FALSE
7:12	LE	<=	FALSE
7:15	IDENT	high	FALSE
7:20	AND	AND	FALSE
7:24	IDENT	found	FALSE
7:30	EQ	==	FALSE
7:33	FALSE	FALSE	FALSE
7:38	RPAREN	)	FALSE
7:40	LBRACE	{	FALSE

Continued on next page

Line:column	Type	Lexeme	Newline before
8:5	IDENT	mid	TRUE
8:9	ASSIGN	=	FALSE
8:11	IDENT	low	FALSE
8:15	PLUS	+	FALSE
8:17	LPAREN	(	FALSE
8:18	IDENT	high	FALSE
8:23	MINUS	-	FALSE
8:25	IDENT	low	FALSE
8:28	RPAREN	)	FALSE
8:30	DIV	DIV	FALSE
8:34	INTEGER	2	FALSE
9:5	LINE_COMMENT	// Update search boun...	TRUE
10:5	KW_IF	IF	TRUE
10:8	LPAREN	(	FALSE
10:9	IDENT	numbers	FALSE
10:16	LBRACKET	[	FALSE
10:17	IDENT	mid	FALSE
10:20	RBRACKET	]	FALSE
10:22	EQ	==	FALSE
10:25	IDENT	target	FALSE
10:31	RPAREN	)	FALSE
10:33	LBRACE	{	FALSE
11:9	IDENT	found	TRUE
11:15	ASSIGN	=	FALSE
11:17	TRUE	TRUE	FALSE
12:9	KW_OUTPUT	OUTPUT	TRUE
12:16	IDENT	mid	FALSE
13:5	RBRACE	}	TRUE
14:5	KW_ELSE	ELSE	TRUE
14:10	KW_IF	IF	FALSE
14:13	LPAREN	(	FALSE
14:14	IDENT	numbers	FALSE
14:21	LBRACKET	[	FALSE
14:22	IDENT	mid	FALSE
14:25	RBRACKET	]	FALSE
14:27	LT	<	FALSE
14:29	IDENT	target	FALSE
14:35	RPAREN	)	FALSE
14:37	LBRACE	{	FALSE
15:9	IDENT	low	TRUE
15:13	ASSIGN	=	FALSE
15:15	IDENT	mid	FALSE
15:19	PLUS	+	FALSE
15:21	INTEGER	1	FALSE
16:5	RBRACE	}	TRUE
17:5	KW_ELSE	ELSE	TRUE
17:10	LBRACE	{	FALSE
18:9	IDENT	high	TRUE
18:14	ASSIGN	=	FALSE
18:16	IDENT	mid	FALSE
18:20	MINUS	-	FALSE
18:22	INTEGER	1	FALSE
19:5	RBRACE	}	TRUE
20:1	RBRACE	}	TRUE
21:1	EOF		TRUE

B Complete Unified DFA Transition Specification

This appendix exhaustively specifies the transition table built by `lalgo_unified.c`: 254 explicit states and 83 input classes, including virtual EOF. It is generated from the implementation during the paper build. The overview in Section 2.3.1 and the smaller category diagrams are readable projections; the tables here supply every concrete state and defined transition.

Each uppercase and lowercase letter is a distinct class (52 classes). DIGIT denotes 0–9. The next 27 classes identify underscore, dot, assignment/equality, exclamation mark, comparisons, slashes, quotes, backslash, whitespace, arithmetic symbols, brackets and punctuation, in their named order. OTHER denotes remaining printable ASCII; ILLEGAL denotes unsupported bytes; EOF is virtual. The alphabet mapping follows the character-set section. All omitted entries are $\delta(q, c) = q_\perp$; the implicit non-accepting sink has $\delta(q_\perp, c) = q_\perp$ for every class. Thus the mathematical transition function is total, although the implementation stores missing transitions as -1 and stops lookahead there.

NONE means non-accepting; SKIP is an accepting whitespace action; every token name labels an accepting state, including INVALID for malformed-number candidates. “Suspected” sets the capitalization diagnostic. “Committed” requires an error when scanning stops before a closing quote or block-comment terminator, even if a shorter prefix was accepted. EOF is emitted by the scanner when the source is exhausted, not by an accepting DFA state. Token emission and restarting at q_0 are scanner actions, not epsilon transitions.

B.1 All Transitions from the Single Start State

Destination	Input classes from q_0
q_{62}	A
q_{48}	B
q_{30}	C
q_{16}	D
q_{86}	E
q_{104}	F
q_1	G, H, J, K, L, M, P, Q, U, V, X, Y, Z, g, h, j, k, l, m, p, q, u, v, x, y, z
q_2	I
q_{154}	N
q_{116}	O
q_{176}	R
q_{72}	S
q_{128}	T
q_{94}	W
q_{63}	a
q_{49}	b
q_{31}	c
q_{17}	d
q_{87}	e
q_{105}	f
q_3	i
q_{155}	n
q_{117}	o
q_{177}	r
q_{73}	s
q_{129}	t
q_{95}	w
q_{216}	DIGIT
q_{239}	DOT
q_{220}	EQ

Destination	Input classes from q_0
q_{222}	BANG
q_{224}	LT
q_{226}	GT
q_{228}	SLASH
q_{242}	STAR
q_{233}	DQUOTE
q_{235}	SQUOTE
q_{238}	NEWLINE, SPACE, TAB
q_{240}	PLUS
q_{241}	MINUS
q_{243}	PERCENT
q_{244}	CARET
q_{245}	LPAREN
q_{246}	RPAREN
q_{247}	LBRACKET
q_{248}	RBRACKET
q_{249}	LBRACE
q_{250}	RBRACE
q_{251}	COMMA
q_{252}	SEMICOLON
q_{253}	COLON

B.2 All States, Accepting Actions, and Remaining Transitions

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_0 `START`	NONE	See the start-state table above.
q_1 `ID`	IDENT	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_2 `E:I`	IDENT	A–Z except F,N, a–z except f,n, DIGIT, UNDERSCORE $\rightarrow q_1$; F $\rightarrow q_{84}$; N $\rightarrow q_4$; f $\rightarrow q_{85}$; n $\rightarrow q_5$
q_3 `F:I`	IDENT	A–Z except F,N, a–z except f,n, DIGIT, UNDERSCORE $\rightarrow q_1$; F, f $\rightarrow q_{85}$; N, n $\rightarrow q_5$
q_4 `E:IN`	IDENT	A–Z except P,T,V, a–z except p,t,v, DIGIT, UNDERSCORE $\rightarrow q_1$; P $\rightarrow q_{110}$; T $\rightarrow q_6$; V $\rightarrow q_{194}$; p $\rightarrow q_{111}$; t $\rightarrow q_7$; v $\rightarrow q_{195}$
q_5 `F:IN`	IDENT	A–Z except P,T,V, a–z except p,t,v, DIGIT, UNDERSCORE $\rightarrow q_1$; P, p $\rightarrow q_{111}$; T, t $\rightarrow q_7$; V, v $\rightarrow q_{195}$
q_6 `E:INT`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E $\rightarrow q_8$; e $\rightarrow q_9$

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_7 `F:INT`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_9
q_8 `E:INTE`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE → q_1; G → q_{10}; g → q_{11}
q_9 `F:INTE`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE → q_1; G, g → q_{11}
q_{10} `E:INTEG`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{12}; e → q_{13}
q_{11} `F:INTEG`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{13}
q_{12} `E:INTEGE`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{14}; r → q_{15}
q_{13} `F:INTEGE`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{15}
q_{14} `E:INTEGER`	KW_INTEGER	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{15} `F:INTEGER`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{16} `E:D`	IDENT	A–Z except E,I, a–z except e,i, DIGIT, UNDERSCORE → q_1; E → q_{18}; I → q_{144}; e → q_{19}; i → q_{145}
q_{17} `F:D`	IDENT	A–Z except E,I, a–z except e,i, DIGIT, UNDERSCORE → q_1; E, e → q_{19}; I, i → q_{145}
q_{18} `E:DE`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C → q_{20}; c → q_{21}
q_{19} `F:DE`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C, c → q_{21}
q_{20} `E:DEC`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I → q_{22}; i → q_{23}
q_{21} `F:DEC`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I, i → q_{23}
q_{22} `E:DECI`	IDENT	A–Z except M, a–z except m, DIGIT, UNDERSCORE → q_1; M → q_{24}; m → q_{25}
q_{23} `F:DECI`	IDENT	A–Z except M, a–z except m, DIGIT, UNDERSCORE → q_1; M, m → q_{25}
q_{24} `E:DECIM`	IDENT	A → q_{26}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{27}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{25} `F:DECIM`	IDENT	A, a → q_{27}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{26} `E:DECIMA`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L → q_{28}; l → q_{29}
q_{27} `F:DECIMA`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L, l → q_{29}
q_{28} `E:DECIMAL`	KW_DECIMAL	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{29} `F:DECIMAL`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{30} `E:C`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE → q_1; H → q_{32}; h → q_{33}
q_{31} `F:C`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE → q_1; H, h → q_{33}
q_{32} `E:CH`	IDENT	A → q_{34}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{35}
q_{33} `F:CH`	IDENT	A, a → q_{35}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{34} `E:CHA`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{36}; r → q_{37}
q_{35} `F:CHA`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{37}
q_{36} `E:CHAR`	IDENT	A → q_{38}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{39}
q_{37} `F:CHAR`	IDENT	A, a → q_{39}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{38} `E:CHARA`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C → q_{40}; c → q_{41}
q_{39} `F:CHARA`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C, c → q_{41}
q_{40} `E:CHARAC`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{42}; t → q_{43}
q_{41} `F:CHARAC`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{43}
q_{42} `E:CHARACT`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{44}; e → q_{45}
q_{43} `F:CHARACT`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{45}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{44} `E:CHARACTE`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{46}; r → q_{47}
q_{45} `F:CHARACTE`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{47}
q_{46} `E:CHARACTER`	KW_CHARACTER	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{47} `F:CHARACTER`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{48} `E:B`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O → q_{50}; o → q_{51}
q_{49} `F:B`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O, o → q_{51}
q_{50} `E:BO`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O → q_{52}; o → q_{53}
q_{51} `F:BO`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O, o → q_{53}
q_{52} `E:BOO`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L → q_{54}; l → q_{55}
q_{53} `F:BOO`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L, l → q_{55}
q_{54} `E:BOOL`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{56}; e → q_{57}
q_{55} `F:BOOL`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{57}
q_{56} `E:BOOLE`	IDENT	A → q_{58}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{59}
q_{57} `F:BOOLE`	IDENT	A, a → q_{59}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{58} `E:BOOLEA`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N → q_{60}; n → q_{61}
q_{59} `F:BOOLEA`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N, n → q_{61}
q_{60} `E:BOOLEAN`	KW_BOOLEAN	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{61} `F:BOOLEAN`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{62} `E:A`	IDENT	A–Z except L,N,R, a–z except l,n,r, DIGIT, UNDERSCORE → q_1; L → q_{160}; N → q_{148}; R → q_{64}; l → q_{161}; n → q_{149}; r → q_{65}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{63} `F:A`	IDENT	A–Z except L,N,R, a–z except l,n,r, DIGIT, UNDERSCORE → q_1; L, l → q_{161}; N, n → q_{149}; R, r → q_{65}
q_{64} `E:AR`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{66}; r → q_{67}
q_{65} `F:AR`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{67}
q_{66} `E:ARR`	IDENT	A → q_{68}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{69}
q_{67} `F:ARR`	IDENT	A, a → q_{69}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{68} `E:ARRA`	IDENT	A–Z except Y, a–z except y, DIGIT, UNDERSCORE → q_1; Y → q_{70}; y → q_{71}
q_{69} `F:ARRA`	IDENT	A–Z except Y, a–z except y, DIGIT, UNDERSCORE → q_1; Y, y → q_{71}
q_{70} `E:ARRAY`	KW_ARRAY	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{71} `F:ARRAY`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{72} `E:S`	IDENT	A–Z except E,T, a–z except e,t, DIGIT, UNDERSCORE → q_1; E → q_{208}; T → q_{74}; e → q_{209}; t → q_{75}
q_{73} `F:S`	IDENT	A–Z except E,T, a–z except e,t, DIGIT, UNDERSCORE → q_1; E, e → q_{209}; T, t → q_{75}
q_{74} `E:ST`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{76}; r → q_{77}
q_{75} `F:ST`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{77}
q_{76} `E:STR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I → q_{78}; i → q_{79}
q_{77} `F:STR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I, i → q_{79}
q_{78} `E:STRI`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N → q_{80}; n → q_{81}
q_{79} `F:STRI`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N, n → q_{81}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{80} `E:STRIN`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE $\rightarrow q_1$; G $\rightarrow q_{82}$; g $\rightarrow q_{83}$
q_{81} `F:STRIN`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE $\rightarrow q_1$; G, g $\rightarrow q_{83}$
q_{82} `E:STRING`	KW_STRING	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{83} `F:STRING`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{84} `E:IF`	KW_IF	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{85} `F:IF`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{86} `E:E`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE $\rightarrow q_1$; L $\rightarrow q_{88}$; l $\rightarrow q_{89}$
q_{87} `F:E`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE $\rightarrow q_1$; L, l $\rightarrow q_{89}$
q_{88} `E:EL`	IDENT	A–Z except S, a–z except s, DIGIT, UNDERSCORE $\rightarrow q_1$; S $\rightarrow q_{90}$; s $\rightarrow q_{91}$
q_{89} `F:EL`	IDENT	A–Z except S, a–z except s, DIGIT, UNDERSCORE $\rightarrow q_1$; S, s $\rightarrow q_{91}$
q_{90} `E:ELS`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E $\rightarrow q_{92}$; e $\rightarrow q_{93}$
q_{91} `F:ELS`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E, e $\rightarrow q_{93}$
q_{92} `E:ELSE`	KW_ELSE	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{93} `F:ELSE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{94} `E:W`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE $\rightarrow q_1$; H $\rightarrow q_{96}$; h $\rightarrow q_{97}$
q_{95} `F:W`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE $\rightarrow q_1$; H, h $\rightarrow q_{97}$
q_{96} `E:WH`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE $\rightarrow q_1$; I $\rightarrow q_{98}$; i $\rightarrow q_{99}$
q_{97} `F:WH`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE $\rightarrow q_1$; I, i $\rightarrow q_{99}$
q_{98} `E:WHI`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE $\rightarrow q_1$; L $\rightarrow q_{100}$; l $\rightarrow q_{101}$
q_{99} `F:WHI`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE $\rightarrow q_1$; L, l $\rightarrow q_{101}$
q_{100} `E:WHIL`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E $\rightarrow q_{102}$; e $\rightarrow q_{103}$

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{101} `F:WHIL`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{103}
q_{102} `E:WHILE`	KW_WHILE	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{103} `F:WHILE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{104} `E:F`	IDENT	A → q_{136}; A–Z except A,O, a–z except a,o, DIGIT, UNDERSCORE → q_1; O → q_{106}; a → q_{137}; o → q_{107}
q_{105} `F:F`	IDENT	A, a → q_{137}; A–Z except A,O, a–z except a,o, DIGIT, UNDERSCORE → q_1; O, o → q_{107}
q_{106} `E:FO`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{108}; r → q_{109}
q_{107} `F:FO`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{109}
q_{108} `E:FOR`	KW_FOR	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{109} `F:FOR`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{110} `E:INP`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U → q_{112}; u → q_{113}
q_{111} `F:INP`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U, u → q_{113}
q_{112} `E:INPU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{114}; t → q_{115}
q_{113} `F:INPU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{115}
q_{114} `E:INPUT`	KW_INPUT	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{115} `F:INPUT`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{116} `E:O`	IDENT	A–Z except R,U, a–z except r,u, DIGIT, UNDERSCORE → q_1; R → q_{152}; U → q_{118}; r → q_{153}; u → q_{119}
q_{117} `F:O`	IDENT	A–Z except R,U, a–z except r,u, DIGIT, UNDERSCORE → q_1; R, r → q_{153}; U, u → q_{119}
q_{118} `E:OU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{120}; t → q_{121}
q_{119} `F:OU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{121}
q_{120} `E:OUT`	IDENT	A–Z except P, a–z except p, DIGIT, UNDERSCORE → q_1; P → q_{122}; p → q_{123}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{121} `F:OUT`	IDENT	A–Z except P, a–z except p, DIGIT, UNDERSCORE → q_1; P, p → q_{123}
q_{122} `E:OUTP`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U → q_{124}; u → q_{125}
q_{123} `F:OUTP`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U, u → q_{125}
q_{124} `E:OUTPU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{126}; t → q_{127}
q_{125} `F:OUTPU`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{127}
q_{126} `E:OUTPUT`	KW_OUTPUT	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{127} `F:OUTPUT`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{128} `E:T`	IDENT	A–Z except H,R, a–z except h,r, DIGIT, UNDERSCORE → q_1; H → q_{212}; R → q_{130}; h → q_{213}; r → q_{131}
q_{129} `F:T`	IDENT	A–Z except H,R, a–z except h,r, DIGIT, UNDERSCORE → q_1; H, h → q_{213}; R, r → q_{131}
q_{130} `E:TR`	IDENT	A → q_{188}; A–Z except A,U, a–z except a,u, DIGIT, UNDERSCORE → q_1; U → q_{132}; a → q_{189}; u → q_{133}
q_{131} `F:TR`	IDENT	A, a → q_{189}; A–Z except A,U, a–z except a,u, DIGIT, UNDERSCORE → q_1; U, u → q_{133}
q_{132} `E:TRU`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{134}; e → q_{135}
q_{133} `F:TRU`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{135}
q_{134} `E:TRUE`	TRUE	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{135} `F:TRUE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{136} `E:FA`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L → q_{138}; l → q_{139}
q_{137} `F:FA`	IDENT	A–Z except L, a–z except l, DIGIT, UNDERSCORE → q_1; L, l → q_{139}
q_{138} `E:FAL`	IDENT	A–Z except S, a–z except s, DIGIT, UNDERSCORE → q_1; S → q_{140}; s → q_{141}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{139} `F:FAL`	IDENT	A–Z except S, a–z except s, DIGIT, UNDERSCORE $\rightarrow q_1$; S, s $\rightarrow q_{141}$
q_{140} `E:FALS`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E $\rightarrow q_{142}$; e $\rightarrow q_{143}$
q_{141} `F:FALS`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE $\rightarrow q_1$; E, e $\rightarrow q_{143}$
q_{142} `E:FALSE`	FALSE	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{143} `F:FALSE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{144} `E:DI`	IDENT	A–Z except V, a–z except v, DIGIT, UNDERSCORE $\rightarrow q_1$; V $\rightarrow q_{146}$; v $\rightarrow q_{147}$
q_{145} `F:DI`	IDENT	A–Z except V, a–z except v, DIGIT, UNDERSCORE $\rightarrow q_1$; V, v $\rightarrow q_{147}$
q_{146} `E:DIV`	DIV	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{147} `F:DIV`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{148} `E:AN`	IDENT	A–Z except D, a–z except d, DIGIT, UNDERSCORE $\rightarrow q_1$; D $\rightarrow q_{150}$; d $\rightarrow q_{151}$
q_{149} `F:AN`	IDENT	A–Z except D, a–z except d, DIGIT, UNDERSCORE $\rightarrow q_1$; D, d $\rightarrow q_{151}$
q_{150} `E:AND`	AND	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{151} `F:AND`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{152} `E:OR`	OR	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{153} `F:OR`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{154} `E:N`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE $\rightarrow q_1$; O $\rightarrow q_{156}$; o $\rightarrow q_{157}$
q_{155} `F:N`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE $\rightarrow q_1$; O, o $\rightarrow q_{157}$
q_{156} `E:NO`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE $\rightarrow q_1$; T $\rightarrow q_{158}$; t $\rightarrow q_{159}$
q_{157} `F:NO`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE $\rightarrow q_1$; T, t $\rightarrow q_{159}$
q_{158} `E:NOT`	NOT	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{159} `F:NOT`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE $\rightarrow q_1$
q_{160} `E:AL`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE $\rightarrow q_1$; G $\rightarrow q_{162}$; g $\rightarrow q_{163}$
q_{161} `F:AL`	IDENT	A–Z except G, a–z except g, DIGIT, UNDERSCORE $\rightarrow q_1$; G, g $\rightarrow q_{163}$
q_{162} `E:ALG`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE $\rightarrow q_1$; O $\rightarrow q_{164}$; o $\rightarrow q_{165}$

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{163} `F:ALG`	IDENT	A–Z except O, a–z except o, DIGIT, UNDERSCORE → q_1; O, o → q_{165}
q_{164} `E:ALGO`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{166}; r → q_{167}
q_{165} `F:ALGO`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{167}
q_{166} `E:ALGOR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I → q_{168}; i → q_{169}
q_{167} `F:ALGOR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I, i → q_{169}
q_{168} `E:ALGORI`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{170}; t → q_{171}
q_{169} `F:ALGORI`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{171}
q_{170} `E:ALGORIT`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE → q_1; H → q_{172}; h → q_{173}
q_{171} `F:ALGORIT`	IDENT	A–Z except H, a–z except h, DIGIT, UNDERSCORE → q_1; H, h → q_{173}
q_{172} `E:ALGORITH`	IDENT	A–Z except M, a–z except m, DIGIT, UNDERSCORE → q_1; M → q_{174}; m → q_{175}
q_{173} `F:ALGORITH`	IDENT	A–Z except M, a–z except m, DIGIT, UNDERSCORE → q_1; M, m → q_{175}
q_{174} `E:ALGORITHM`	RES_ALGORITHM	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{175} `F:ALGORITHM`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{176} `E:R`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{178}; e → q_{179}
q_{177} `F:R`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{179}
q_{178} `E:RE`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{180}; t → q_{181}
q_{179} `F:RE`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{181}
q_{180} `E:RET`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U → q_{182}; u → q_{183}
q_{181} `F:RET`	IDENT	A–Z except U, a–z except u, DIGIT, UNDERSCORE → q_1; U, u → q_{183}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{182} `E:RETU`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{184}; r → q_{185}
q_{183} `F:RETU`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{185}
q_{184} `E:RETUR`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N → q_{186}; n → q_{187}
q_{185} `F:RETUR`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N, n → q_{187}
q_{186} `E:RETURN`	RES_RETURN	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{187} `F:RETURN`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{188} `E:TRA`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C → q_{190}; c → q_{191}
q_{189} `F:TRA`	IDENT	A–Z except C, a–z except c, DIGIT, UNDERSCORE → q_1; C, c → q_{191}
q_{190} `E:TRAC`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{192}; e → q_{193}
q_{191} `F:TRAC`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{193}
q_{192} `E:TRACE`	RES_TRACE	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{193} `F:TRACE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{194} `E:INV`	IDENT	A → q_{196}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{197}
q_{195} `F:INV`	IDENT	A, a → q_{197}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1
q_{196} `E:INVA`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R → q_{198}; r → q_{199}
q_{197} `F:INVA`	IDENT	A–Z except R, a–z except r, DIGIT, UNDERSCORE → q_1; R, r → q_{199}
q_{198} `E:INVAR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I → q_{200}; i → q_{201}
q_{199} `F:INVAR`	IDENT	A–Z except I, a–z except i, DIGIT, UNDERSCORE → q_1; I, i → q_{201}
q_{200} `E:INVARI`	IDENT	A → q_{202}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1; a → q_{203}
q_{201} `F:INVARI`	IDENT	A, a → q_{203}; A–Z except A, a–z except a, DIGIT, UNDERSCORE → q_1

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{202} `E:INVARIA`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N → q_{204}; n → q_{205}
q_{203} `F:INVARIA`	IDENT	A–Z except N, a–z except n, DIGIT, UNDERSCORE → q_1; N, n → q_{205}
q_{204} `E:INVARIAN`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{206}; t → q_{207}
q_{205} `F:INVARIAN`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{207}
q_{206} `E:INVARIANT`	RES_INVARIANT	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{207} `F:INVARIANT`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{208} `E:SE`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T → q_{210}; t → q_{211}
q_{209} `F:SE`	IDENT	A–Z except T, a–z except t, DIGIT, UNDERSCORE → q_1; T, t → q_{211}
q_{210} `E:SET`	NOISE_SET	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{211} `F:SET`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{212} `E:TH`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E → q_{214}; e → q_{215}
q_{213} `F:TH`	IDENT	A–Z except E, a–z except e, DIGIT, UNDERSCORE → q_1; E, e → q_{215}
q_{214} `E:THE`	NOISE_THE	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{215} `F:THE`	IDENT; suspected	A–Z, a–z, DIGIT, UNDERSCORE → q_1
q_{216} `INT`	INTEGER	A–Z, a–z, UNDERSCORE → q_{219}; DIGIT → q_{216}; DOT → q_{217}
q_{217} `INT.`	NONE	DIGIT → q_{218}
q_{218} `DEC`	DECIMAL	A–Z, a–z, UNDERSCORE → q_{219}; DIGIT → q_{218}
q_{219} `BAD_NUM`	INVALID	A–Z, a–z, DIGIT, UNDERSCORE → q_{219}
q_{220} `ASSIGN`	ASSIGN	EQ → q_{221}
q_{221} `EQ`	EQ	None (all inputs stop the match).
q_{222} `BANG`	NONE	EQ → q_{223}
q_{223} `NEQ`	NEQ	None (all inputs stop the match).
q_{224} `LT`	LT	EQ → q_{225}
q_{225} `LE`	LE	None (all inputs stop the match).
q_{226} `GT`	GT	EQ → q_{227}
q_{227} `GE`	GE	None (all inputs stop the match).
q_{228} `SLASH`	SLASH	SLASH → q_{229}; STAR → q_{230}
q_{229} `LINE_CMT`	LINE_COMMENT	all real classes except NEWLINE → q_{229}
q_{230} `BLK_CMT`	NONE; committed comment	all real classes except STAR → q_{230}; STAR → q_{231}
q_{231} `BLK_STAR`	NONE; committed comment	all real classes except SLASH, STAR → q_{230}; SLASH → q_{232}; STAR → q_{231}
q_{232} `BLK_END`	BLOCK_COMMENT	None (all inputs stop the match).
q_{233} `STR_OPEN`	NONE; committed string	all real classes except DQUOTE, BACKSLASH, NEWLINE, ILLEGAL → q_{233}; DQUOTE → q_{234}

Continued on next page

State / implementation name	Accepting action / metadata	Defined outgoing transitions
q_{234} `STR_DONE`	STRING	None (all inputs stop the match).
q_{235} `CHR_OPEN`	NONE; committed character	all real classes except SQUOTE, BACKSLASH, NEWLINE, ILLEGAL $\rightarrow q_{236}$
q_{236} `CHR_BODY`	NONE; committed character	SQUOTE $\rightarrow q_{237}$
q_{237} `CHR_DONE`	CHAR	None (all inputs stop the match).
q_{238} `WS`	SKIP	NEWLINE, SPACE, TAB $\rightarrow q_{238}$
q_{239} `DOT`	DOT	None (all inputs stop the match).
q_{240} `PLUS`	PLUS	None (all inputs stop the match).
q_{241} `MINUS`	MINUS	None (all inputs stop the match).
q_{242} `STAR`	STAR	None (all inputs stop the match).
q_{243} `PERCENT`	PERCENT	None (all inputs stop the match).
q_{244} `CARET`	CARET	None (all inputs stop the match).
q_{245} `LPAREN`	LPAREN	None (all inputs stop the match).
q_{246} `RPAREN`	RPAREN	None (all inputs stop the match).
q_{247} `LBRACKET`	LBRACKET	None (all inputs stop the match).
q_{248} `RBRACKET`	RBRACKET	None (all inputs stop the match).
q_{249} `LBRACE`	LBRACE	None (all inputs stop the match).
q_{250} `RBRACE`	RBRACE	None (all inputs stop the match).
q_{251} `COMMA`	COMMA	None (all inputs stop the match).
q_{252} `SEMICOLON`	SEMICOLON	None (all inputs stop the match).
q_{253} `COLON`	COLON	None (all inputs stop the match).